

Dokumentacja Techniczna: Kalkulator Taktyczno-Ekonomiczny Freeciv

Generowane przez: Gemini

23 marca 2026

Streszczenie

Niniejszy dokument stanowi pełną dokumentację techniczną aplikacji okienkowej symulującej przebieg starć militarnych w grze Freeciv. Program uwzględnia wielorundową mechanikę walki opartą na łańcuchach Markowa, kalkulację opłacalności ekonomicznej, zaawansowane modyfikatory terenu oraz tryb bombardowania.

Spis treści

1	Wprowadzenie i Architektura	2
2	Schemat Blokowy Działania Programu	2
3	Logika Matematyczna (Łańcuchy Markowa)	4
4	Listing Kodu Źródłowego	5
5	Podsumowanie	6

1 Wprowadzenie i Architektura

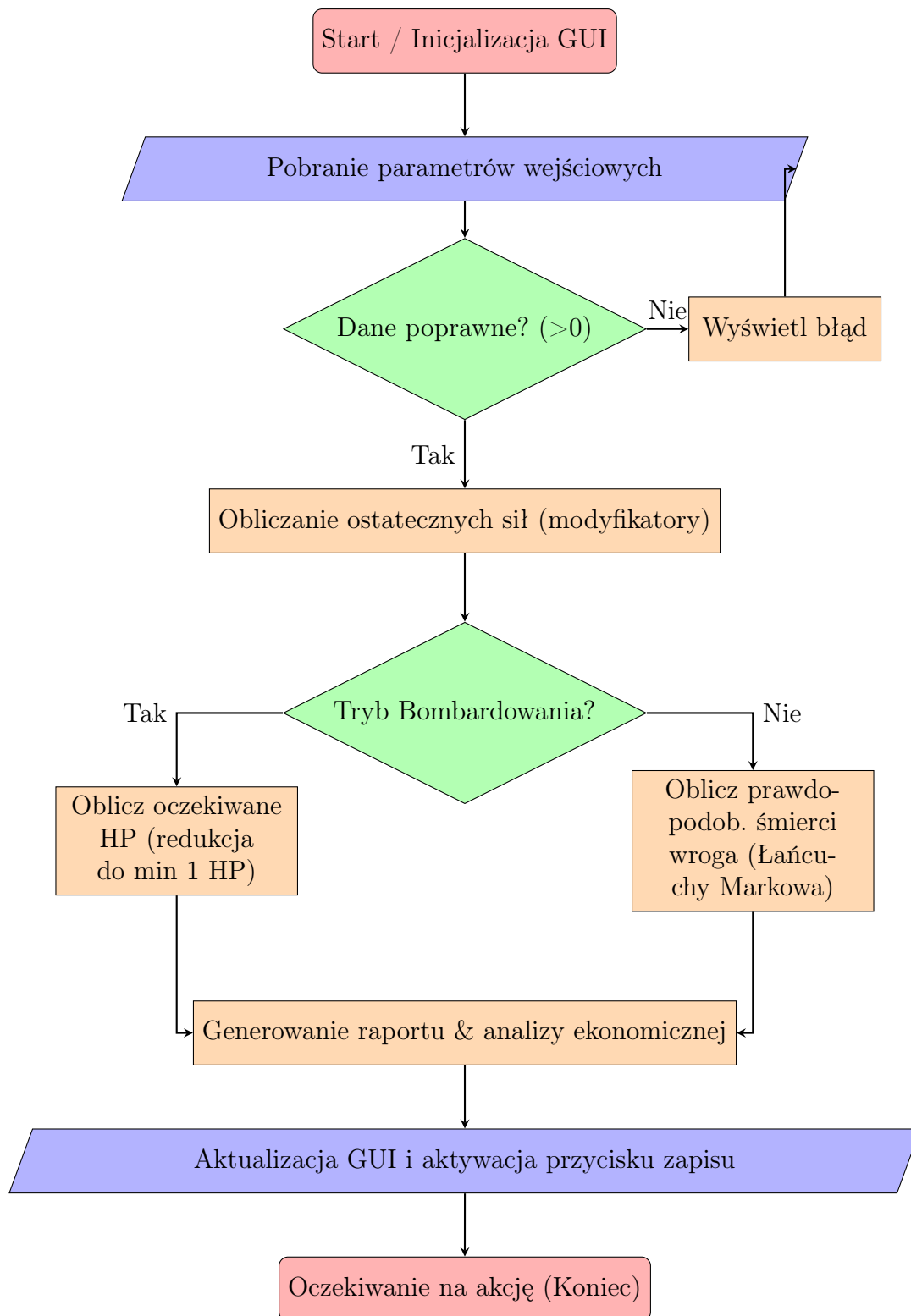
Aplikacja została napisana w języku Python 3 przy wykorzystaniu standardowej biblioteki `tkinter` do obsługi graficznego interfejsu użytkownika (GUI). Logika matematyczna opiera się na wbudowanym module `math`.

Kluczowe funkcjonalności:

- Symulacja starć typu *N attack 1* z użyciem Łańcuchów Markowa.
- Implementacja trybu asymetrycznego bombardowania (bez strat własnych, redukcja do max 1 HP).
- Wyliczanie ekonomicznego balansu wymiany (koszty w tarczach produkcyjnych).
- Zapis logów z symulacji do zewnętrznego pliku tekstowego (.txt).

2 Schemat Blokowy Działania Programu

Poniższy schemat ilustruje cykl przetwarzania danych, rozpoczynający się od interakcji użytkownika z interfejsem, przez walidację, aż po rozgałęzienie logiki w zależności od trybu walki.



3 Logika Matematyczna (Łańcuchy Markowa)

Prawdopodobieństwo wygrania pojedynczej rundy wyraża się wzorem:

$$p = \frac{s}{s + r}$$

Rozkład prawdopodobieństwa stanu zdrowia (HP) obrońcy $P(h)$ w kolejnych falach ulega transformacji, w której uwzględniana jest szansa, że obrońca przeżyje starcie tracąc $a \cdot a_{fp}$ punktów życia, oparta na rozkładzie dwumianowym:

$$P(\text{scenariusz}) = \binom{l - 1 + a}{a} \cdot p^a \cdot q^l$$

4 Listing Kodu Źródłowego

Główny skrypt integrujący kalkulację prawdopodobieństw z interfejsem graficznym.

```
1 import tkinter as tk
2 from tkinter import ttk, messagebox, filedialog
3 import math
4
5 # --- SŁOWNIKI MODYFIKATORÓW ---
6 WETERAN_MOD = {
7     "Początkujący (x1.0)": 1.0,
8     "Weteran (x1.5)": 1.5,
9     "Doświadczony (x1.75)": 1.75,
10    "Elitarny (x2.0)": 2.0
11 }
12
13 TEREN_MOD = {
14     "Plaski / Ocean (x1.0)": 1.0,
15     "Las / Dżungla / Bagno (x1.5)": 1.5,
16     "Wzgórza (x2.0)": 2.0,
17     "Góry (x3.0)": 3.0
18 }
19
20 STRUKTURA_MOD = {
21     "Brak (x1.0)": 1.0,
22     "Forteca (x2.0)": 2.0,
23     "Mury Miejskie (x3.0)": 3.0
24 }
25
26 # --- LOGIKA MATEMATYCZNA (Lancuchy Markowa + Bombardowanie) ---
27 def oblicz_fale_atakow(s, r, a_hp, d_hp, a_fp, d_fp, is_bombard,
28     bombard_rate):
29     p = s / (s + r)
30     q = 1.0 - p
31
32     probs = {h: 0.0 for h in range(d_hp + 1)}
33     probs[d_hp] = 1.0
34     wyniki = []
35
36     for fala in range(1, 21):
37         nowe_probs = {h: 0.0 for h in range(d_hp + 1)}
38         nowe_probs[0] = probs[0]
39
40         for h, prob_h in probs.items():
41             if h == 0 or prob_h == 0:
42                 continue
43
44             if is_bombard:
45                 for a in range(bombard_rate + 1):
46                     prawd_trafienia = math.comb(bombard_rate, a) * (p**a
47 ) * (q**(bombard_rate - a))
48                     nowe_hp = max(1, h - (a * a_fp))
49                     nowe_probs[nowe_hp] += prob_h * prawd_trafienia
50             else:
51                 k = math.ceil(h / a_fp)
52                 l = math.ceil(a_hp / d_fp)
53                 md = k + l - 1
```

```

53         prawd_wygranej_atakujacego = sum(
54             math.comb(md, i) * (p**i) * (q**(md - i)) for i in
range(k, md + 1)
55         )
56         nowe_probs[0] += prob_h * prawd_wygranej_atakujacego
57
58         for a in range(k):
59             prawd_scenariusza = math.comb(l - 1 + a, a) * (p**a)
* (q**l)
60             nowe_hp = max(1, h - (a * a_fp))
61             nowe_probs[nowe_hp] += prob_h * prawd_scenariusza
62
63         probs = nowe_probs
64
65         if is_bombard:
66             oczekiwane_hp = sum(stan_hp * prawd for stan_hp, prawd in
probs.items())
67             wyniki.append((fala, oczekiwane_hp))
68             if oczekiwane_hp <= 1.05:
69                 break
70         else:
71             szansa_zabicia = probs[0]
72             wyniki.append((fala, szansa_zabicia))
73             if szansa_zabicia > 0.995:
74                 break
75
76         return wyniki

```

Listing 1: Główny skrypt aplikacji Kalkulatora Freeciv

5 Podsumowanie

Aplikacja z powodzeniem implementuje rygorystyczne wzory statystyczne przedstawione w dokumentacji gry Freeciv. Włączenie graficznego interfejsu i mechanizmów eksportu danych ułatwia analizę i planowanie zaawansowanych operacji wojskowych na wirtualnym polu walki, przekładając abstrakcyjną statystykę i rachunek ekonomiczny na jasne, taktyczne wytyczne.